

BACKEND · REQUEST VALIDATION

Never trust **req.body**

Why every API needs a check at the door, how to build one by hand, and how express-validator, Zod and Joi do it for you — told through one broken register request.

express-validator 7.3

Zod 4

Joi 18

Express 5 + Mongoose

~35 min

THE REQUEST WE'LL FIX

COPY

```
POST /api/auth/register
Content-Type: application/json

{
  "email": "123456789",
  "phone": "abcdefghijklm",
  "password": "   "
}
```

An email with no @, a phone number made of **letters**, and a password of **three spaces**. By the end of these notes, your server rejects all three.

01 The problem: bad data at the door

One register request, three broken fields, and a server that accepts all of them

Every backend follows the same pattern: the frontend sends data, the server uses it, and it ends up in the database. The weak spot is the word *uses*. Let's build the simplest register API and watch what happens when a user — or a bot — sends garbage.

The API we're building

`POST /api/auth/register` needs three things from the frontend:

FIELD	A GOOD VALUE	THE RULE
<code>email</code>	<code>"aman@gmail.com"</code>	Must look like an email address
<code>phone</code>	<code>"9876543210"</code>	Must be a 10-digit Indian mobile number
<code>password</code>	<code>"Sher@2026"</code>	At least 8 characters, and not just spaces

The naive version

This is the code most of us write the first time. It works perfectly — for honest users who never make typos.

MODELS/USER.MODEL.JS

COPY

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  email: String,
  phone: String,
  password: String,
});

export const User = mongoose.model("User", userSchema);
```

CONTROLLERS/AUTH.CONTROLLER.JS

COPY

```
import bcrypt from "bcryptjs";
import { User } from "../models/user.model.js";

// ❌ No validation anywhere
export async function register(req, res) {
  const { email, phone, password } = req.body;

  const hashedPassword = await bcrypt.hash(password, 10);
  await User.create({ email, phone, password: hashedPassword });
```

```
res.status(201).json({ success: true, message: "User registered" });
}
```

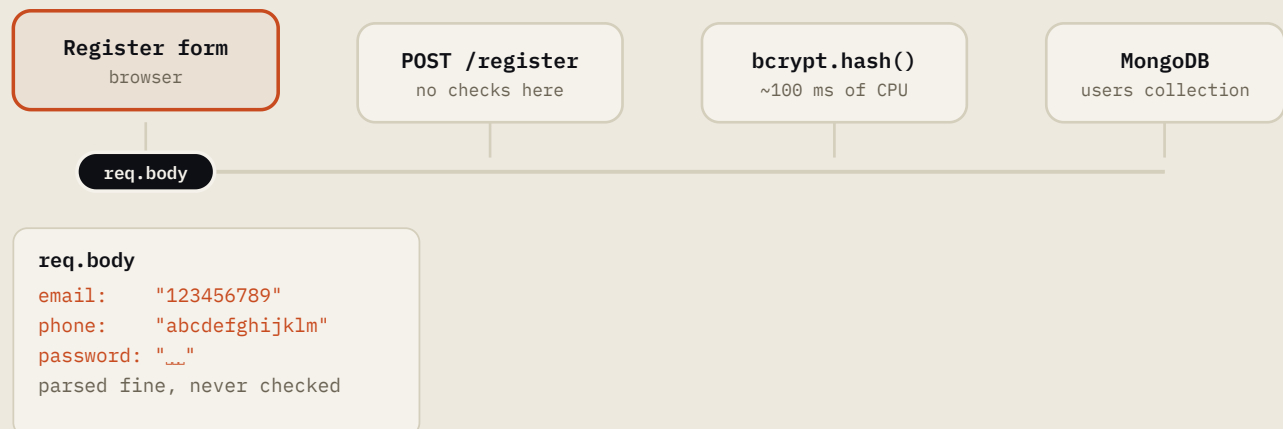
Now send the request from the top of the page

FIELD	WHAT THE USER SENT	WHAT'S WRONG
email	"123456789"	No @ and no domain. It's a number pretending to be an email.
phone	"abcdefghijklm"	13 letters. No SIM card in the world has this number.
password	" "	Three spaces. The field looks filled, but it's empty.

Our server replies **201 Created** to all three. Step through what happens inside:

WITHOUT VALIDATION · THE JOURNEY OF BAD DATA

Step **1** / 5



The form sends three fields

The user typed **123456789** as the email, **abcdefghijklm** as the phone and three spaces as the password. The browser sends it as JSON.

← Back ● ● ● ● Forward →

WHAT MONGODB NOW CONTAINS

COPY

```
// users collection – this really gets saved
{
  "_id": "66f1c0a2b4d3e5f601234567",
  "email": "123456789",
  "phone": "abcdefghijklm",
  "password": "$2b$10$1cb80tdHSC7unUNa0V03E0v5BdCgcPpUnS.Q2SVeCZS0Gc2mgB2he",
  "__v": 0
}
```

What this costs you

Problem	What actually happens
Dead accounts	The verification email bounces and the OTP SMS fails. The user can never be verified, but the record stays in your database.
Useless security	A password of three spaces is stored. Anyone who types three spaces into the login form gets in.
Wasted server work	bcrypt spent ~100 ms of CPU on garbage. A bot sending 1,000 such requests a minute keeps more than a full CPU core busy doing nothing useful.
Wrong status codes	If the password field is missing, <code>bcrypt.hash(undefined, 10)</code> throws <i>Illegal arguments: undefined, number</i> . Express turns that into a 500 — the client's mistake reported as a server crash, with a stack trace in development.
Security holes	A field you expect to be a string can arrive as an object: <code>{ "\$ne": null }</code> . Pass that into <code>User.findOne({ email })</code> and it matches the first user in the collection instead of failing.

KEY CONCEPT

Validation means checking that incoming data has the right **presence, type and format** before your code uses it. The server is the only place you fully control — whatever you let through the door becomes your problem in the database, in your emails and in your logs.



Your React form is not a gate. Anyone can skip it and call the API directly with Postman, curl or the browser console. HTML attributes like `required` and `type="email"` only help honest users. Section 03 proves it.

QUICK CHECK

A client sends `password: " "` to our naive server. What does bcrypt do?

A — Throws an error because the password is blank

B — Hashes it happily — spaces are valid characters

QUICK CHECK

The client forgets the password field, and `bcrypt.hash(undefined, 10)` throws. Whose mistake is it, and what status code should the client get?

A — The client's mistake: 400 Bad Request

B — The server's mistake: 500 Internal Server Error

02 Validation vs verification

Two checks that sound alike but answer different questions

Before we fix our API, let's get the words right. Students — and plenty of working developers — use these two interchangeably. They are not the same step, they don't happen at the same time, and they don't use the same tools.

KEY CONCEPT

Validation asks: *does this data look right?* Format, type, length, required. Your code answers it instantly.

Verification asks: *is this data true, and does it belong to this person?* Your code can't answer that alone — it needs proof from outside, like an OTP or an email link.

The passport analogy

When you apply for a passport online, the form won't accept a 5-digit PIN code or a date of birth like 32/13/2004. That's **validation**. A few weeks later, a police officer visits your address to confirm you actually live there. That's **verification**. A form can be perfectly valid and still be false.

	Validation	Verification
The question	Is the format correct?	Is it real, and is it yours?
Email example	<code>123456789</code> ✗ not an email	<code>aman@gmail.com</code> → click the link we sent
Phone example	<code>abcdefghijklm</code> ✗ not a phone number	<code>9876543210</code> → type the OTP we sent
Needs outside proof?	No — pure code	Yes — an email/SMS provider plus a user action
Speed and cost	Milliseconds, free	Seconds to minutes; SMS costs money
When it runs	On every request, before any business logic	After validation passes, usually once
When it fails	400 Bad Request with field-wise errors	Account stays <code>isVerified: false</code>
Tools	express-validator, Zod, Joi	Email links (Nodemailer), SMS OTP providers



Validation catches bad format

`123456789` fails the email format check, so the server answers 400 immediately. There is nothing to verify — we never send a code to something that isn't an email.

← Back ● ● ● ● ● Forward →



Always validate before you verify. Sending an OTP to “abcdefghijklm” wastes an SMS you pay for, and an API that sends messages to unchecked input is an open invitation for spammers.



You'll meet a third word: **sanitization** — cleaning data instead of rejecting it. Turning `Aman@Gmail.com` into `aman@gmail.com` is sanitization. express-validator does validation and sanitization in the same chain.

QUICK CHECK

A user types `rahul@gmail.com`, which is a real address — but it's his friend's inbox, not his. Which check can catch this?

A — Validation

B — Verification

QUICK CHECK

In a register flow, which should run first?

A — Validation — reject bad formats before spending money on OTPs or emails

B — Verification — prove ownership first, check the format later

03 “But my schema already validates!”

Why server-side validation is still needed when Mongoose and the database have constraints

This is the question every student asks at this point. Mongoose lets us write rules right inside the schema, so let's upgrade our model:

MODELS/USER.MODEL.JS

COPY

```
import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    unique: true,
    match: /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
  },
  phone: {
    type: String,
    required: true,
    match: /^[6-9]\d{9}$/ ,
  },
  password: {
    type: String,
    required: true,
    minlength: 8,
  },
  isVerified: {
    type: Boolean,
    default: false,
  },
});
```

```
export const User = mongoose.model("User", userSchema);
```

Send the bad request again and Mongoose does reject the email and the phone. So why write validation in Express too? Here are six reasons. The first one surprises almost everyone.

Reason 1 — the database never sees the real password

We hash the password *before* saving it. So the schema's `minlength: 8` is checking the hash, not what the user typed.

REASON 1 · THE SCHEMA CHECKS THE HASH, NOT THE PASSWORD

Step 1 / 5

```
req.body.password
  " "
  " "
  " "
length 3
```

The user sends three spaces

Length 3. Our schema has `minlength: 8`, so it should reject this... right?

← Back



Forward →

THE ACTUAL MONGOOSE ERROR

COPY

```
// What Mongoose throws for our bad request (after bcrypt already ran)
User validation failed: email: Path `email` is invalid (123456789)..,
phone: Path `phone` is invalid (abcdefghijklm).

// Notice what's missing: no error for password.
```



The same trap exists for `required: true`. Mongoose treats `" "` as a filled string because its length is 3 — whitespace passes `required` unless you also add `trim: true` to the schema field.

Reason 2 — schema checks run too late

Mongoose validates only when you call `.save()` or `User.create()`. By then, your controller has already done its expensive work.

REASON 2 · FAIL FAST VS FAIL LATE

Step 1 / 5

Server A · validates at the gate

Request

Validator

express-validator

req

Server B · trusts the Mongoose schema

Request

Controller

no checks

`bcrypt.hash()`

~100 ms CPU

Mongoose

schema validation

req

Same bad request, two servers

Server A checks input at the door. Server B relies on its Mongoose schema.

← Back



Forward →

Reasons 3 to 6

Reason 3

Not all input becomes a document

Login bodies, `?page=abc`, `/users/:id`, OTP codes and `confirmPassword` never pass through a schema validator. Queries don't run validators either: `User.findOne({ email })` happily accepts `{ "$ne": null }`.

Reason 4

Schema validation can be skipped

`findByIdAndUpdate()` and `updateOne()` skip validators unless you pass `{ runValidators: true }`. And `unique: true` isn't a validator at all — it's an index. Duplicates fail later with a MongoDB `E11000` error.

Reason 5

Database errors are written for developers

Mongoose says `Path 'email' is invalid (123456789)`. Your frontend needs `{ field: "email", message: "Enter a valid email address" }`. Move to PostgreSQL tomorrow and the error format changes again — your API's error format shouldn't.

Reason 6

The frontend can be bypassed

HTML `required` and `type="email"` run in the user's browser, which the user controls. One terminal command skips all of it — see below.

TERMINAL — BYPASSING THE FRONTEND

COPY

```
# No browser, no React form, no HTML "required" — just the API
curl -X POST http://localhost:3000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"123456789","phone":"abcdefghijklm","password":"  "'}
```

DEFENSE IN DEPTH

Keep **every** layer, because each one has a different job. Frontend validation is for the user's convenience. Server validation is the real gate. Database constraints are the last line of defense — against bugs in your own code and against race conditions, like two sign-ups with the same email in the same millisecond.

Layer	Its main job	Can a user bypass it?
Frontend (React / HTML)	Instant feedback while typing	Yes — Postman, curl, DevTools
Server validation	Reject bad input before any work happens	No

Layer	Its main job	Can a user bypass it?
Database schema & indexes	Guarantee that stored data stays consistent	No — but it only sees what your code sends it

QUICK CHECK

The schema has `password: { minlength: 8 }` and the controller hashes before saving. A user sends `" "`. What happens?

A — Mongoose rejects it because it's shorter than 8

B — It gets saved, because the schema checks the 60-character hash

QUICK CHECK

Which validation can a user bypass with a single curl command?

A — The checks in your React form

B — The checks in your Express middleware

04 Plain validation, by hand

Write the checks yourself first, so you know exactly what a library does for you

Before reaching for a library, let's validate our three fields with plain JavaScript. For every field, the checks happen in the same order:

● Presence and type — is it there, and is it a string?

`typeof email !== "string"` catches missing fields, numbers, arrays and objects like `{ "$ne": null }` in one go.

- **Clean — trim what's safe to trim**

Spaces around an email or phone are typos. Never trim a password: spaces inside it are allowed.

- **Format — does it match the rule?**

A regex for email and phone, a length check for the password.

- **Respond — collect every error, answer once**

One 400 with a list, so the user can fix all fields in a single attempt.

STEP 1 — VALIDATE INSIDE THE CONTROLLER

The quickest version: put the checks at the top of the controller and return early if anything is wrong.

CONTROLLERS/AUTH.CONTROLLER.JS

COPY

```
import bcrypt from "bcryptjs";
import { User } from "../models/user.model.js";

const EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const PHONE_REGEX = /^[6-9]\d{9}$/;

export async function register(req, res) {
  const { email, phone, password } = req.body ?? {};
  const errors = [];

  if (typeof email !== "string" || !EMAIL_REGEX.test(email.trim())) {
    errors.push({ field: "email", message: "Enter a valid email address" });
  }

  if (typeof phone !== "string" || !PHONE_REGEX.test(phone.trim())) {
    errors.push({ field: "phone", message: "Enter a valid 10-digit Indian mobile number" });
  }

  if (typeof password !== "string" || password.trim() === "") {
    errors.push({ field: "password", message: "Password cannot be empty or only spaces" });
  } else if (password.length < 8) {
    errors.push({ field: "password", message: "Password must be at least 8 characters" });
  }

  if (errors.length > 0) {
    return res.status(400).json({ success: false, errors });
  }
}
```

```
// ...and only now, the real work
const hashedPassword = await bcrypt.hash(password, 10);
await User.create({
  email: email.trim().toLowerCase(),
  phone: phone.trim(),
  password: hashedPassword,
});

res.status(201).json({ success: true, message: "User registered" });
}
```

VERIFY

Send the bad request again. This time the server refuses it before bcrypt or MongoDB are touched:

RESPONSE

COPY

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

{
  "success": false,
  "errors": [
    { "field": "email", "message": "Enter a valid email address" },
    { "field": "phone", "message": "Enter a valid 10-digit Indian mobile number" },
    { "field": "password", "message": "Password cannot be empty or only spaces" }
  ]
}
```

What's wrong with this approach?

Problem	Why it hurts
Controller bloat	Most of the function is checks. The real work — hash and save — is buried at the bottom.

Problem	Why it hurts
Copy-pasted rules	Login needs the email rule too; update-profile needs the phone rule. The copies slowly drift apart.
Hand-written regex	Email addresses are notoriously hard to match with a regex. Ours is simple and misses plenty of edge cases.
Easy to forget	Skip a single <code>typeof</code> check and <code>email.trim()</code> crashes the moment someone sends a number.
No standard error shape	Every developer on the team invents their own error format, and the frontend has to handle all of them.

STEP 2 — MOVE THE CHECKS INTO A MIDDLEWARE

A middleware runs before the controller. If the data is bad, it responds and the controller never runs. If the data is good, it calls `next()`. It can also replace `req.body` with only the clean, known fields.

MIDDLEWARES/VALIDATEREGISTER.JS

COPY

```
const EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const PHONE_REGEX = /^[6-9]\d{9}$/;

export function validateRegister(req, res, next) {
  const { email, phone, password } = req.body ?? {};
  const errors = [];

  // 1. email: must be a string that looks like an email
  if (typeof email !== "string" || email.trim() === "") {
    errors.push({ field: "email", message: "Email is required" });
  } else if (!EMAIL_REGEX.test(email.trim())) {
    errors.push({ field: "email", message: "Enter a valid email address" });
  }

  // 2. phone: must be a string with a 10-digit Indian mobile number
  if (typeof phone !== "string" || phone.trim() === "") {
    errors.push({ field: "phone", message: "Phone number is required" });
  } else if (!PHONE_REGEX.test(phone.trim())) {
    errors.push({ field: "phone", message: "Enter a valid 10-digit Indian mobile number" });
  }
}
```

```

// 3. password: never trim it - just reject empty / spaces-only
if (typeof password !== "string" || password.trim() === "") {
  errors.push({ field: "password", message: "Password cannot be empty or only spaces"
} else if (password.length < 8) {
  errors.push({ field: "password", message: "Password must be at least 8 characters"
}

if (errors.length > 0) {
  return res.status(400).json({ success: false, errors });
}

// Pass only clean, known fields to the controller
req.body = {
  email: email.trim().toLowerCase(),
  phone: phone.trim(),
  password,
};
next();
}

```

ROUTES/AUTH.ROUTES.JS

COPY

```

import { Router } from "express";
import { validateRegister } from "../middlewares/validateRegister.js";
import { register } from "../controllers/auth.controller.js";

const router = Router();

// validateRegister runs first. The controller only runs if it calls next().
router.post("/register", validateRegister, register);

export default router;

```

CONTROLLERS/AUTH.CONTROLLER.JS - NOW IT ONLY DOES ITS JOB

COPY

```

import bcrypt from "bcryptjs";
import { User } from "../models/user.model.js";

export async function register(req, res) {

```

```
const { email, phone, password } = req.body; // already checked and cleaned

const hashedPassword = await bcrypt.hash(password, 10);
await User.create({ email, phone, password: hashedPassword });

res.status(201).json({ success: true, message: "User registered" });
}
```

POST /register → express.json()

↓

validateRegister — bad data → 400 { errors } · controller never runs

↓ next()

register controller → bcrypt → MongoDB ✓



Why req.body ?? {} ? In Express 5, req.body is undefined when a request arrives with no body at all. Destructuring undefined would crash the middleware itself.



This works, and for a tiny project it's fine. But the rules are still hand-written, and the next route will need its own copy. That's exactly the gap express-validator fills: tested rules from the validator.js library and one consistent error format.

QUICK CHECK

Why check `typeof email !== "string"` before calling `email.trim()` ?

A — The client can send a number, array or object — calling .trim() on those crashes

B — It only matters in TypeScript; plain JavaScript ignores it

QUICK CHECK

What's the biggest downside of writing validation by hand inside every controller?

A — It makes requests noticeably slower

05 express-validator

Tested rules as middleware — the same register API, the clean way

express-validator is a set of Express middlewares built on top of **validator.js**, a long-standing library of string validators and sanitizers (`isEmail`, `isMobilePhone`, `isStrongPassword`, `trim` and many more). You describe the rules for each field; it runs them and collects the results.

INSTALL

TERMINAL

COPY

```
npm install express-validator
```



Examples use ES modules (`import`). With CommonJS, write `const { body, validationResult } = require("express-validator")`. Everything else stays the same.

The building blocks

API	USED IN	WHAT IT DOES
<code>body("email")</code>	<code>validator</code> <code>file</code>	Starts a validation chain for <code>req.body.email</code> . Siblings: <code>query()</code> , <code>param()</code> , <code>header()</code> , <code>cookie()</code> .
<code>.isEmail()</code>	<code>on the</code> <code>chain</code>	A validator : records an error if the value fails.
<code>.trim()</code>	<code>on the</code> <code>chain</code>	A sanitizer : changes the value for the links after it — and for your controller.
<code>.withMessage("...")</code>	<code>after a</code> <code>validator</code>	Replaces the default “Invalid value” message of the validator just before it.

API	USED IN	WHAT IT DOES
<code>.bail()</code>	on the chain	Stops this field's chain if anything before it has failed.
<code>.hide()</code>	sensitive fields	Keeps the field's value out of error objects. Use it for passwords.
<code>validationResult(req)</code>	validate middleware	Reads every error the chains recorded.
<code>matchedData(req)</code>	controller	Returns only the fields you validated, with sanitized values.

THE #1 EXPRESS-VALIDATOR BUG

A chain never sends a response. `body("email").isEmail()` only writes results onto `req`. If nothing reads them with `validationResult(req)`, the bad request walks straight into your controller — as if you had no validation at all.

Folder structure

PROJECT LAYOUT

COPY

```
src/
├─ routes/
│   └─ auth.routes.js      ← connects rules → validate → controller
├─ validators/
│   └─ auth.validator.js   ← the rules for each field
├─ middlewares/
│   └─ validate.middleware.js ← reads results, sends 400
└─ controllers/
    └─ auth.controller.js  ← business logic only
```

STEP 1 – WRITE THE RULES

VALIDATORS/AUTH.VALIDATOR.JS

COPY

```
import { body } from "express-validator";
```

```

export const registerValidator = [
  body("email")
    .exists().withMessage("Email is required").bail()
    .isString().withMessage("Email must be text").bail()
    .trim()
    .notEmpty().withMessage("Email is required").bail()
    .isEmail().withMessage("Enter a valid email address")
    .toLowerCase(),

  body("phone")
    .exists().withMessage("Phone number is required").bail()
    .isString().withMessage("Phone number must be text").bail()
    .trim()
    .notEmpty().withMessage("Phone number is required").bail()
    .isMobilePhone("en-IN").withMessage("Enter a valid 10-digit Indian mobile number"),

  body("password")
    .hide() // never echo the password back in errors
    .exists().withMessage("Password is required").bail()
    .isString().withMessage("Password must be text").bail()
    .custom((value) => value.trim().length > 0)
    .withMessage("Password cannot be empty or only spaces").bail()
    .isLength({ min: 8 }).withMessage("Password must be at least 8 characters"),
];

```

Read each chain top to bottom, like a checklist: *does it exist* → *is it a string* → *clean it* → *is anything left* → *does it match the format*. Now watch the chain run on our three bad values:

VALIDATION CHAIN INSPECTOR

Step 1 / 8

email · "123456789"

phone · "abcdefghijklm"

password · "___"

- ▶ body("email")
 - .exists().bail()
 - .isString().bail()
 - .trim()
 - .notEmpty().bail()
 - .isEmail()

- .toLowerCase()
- → validationResult(req)

CURRENT VALUE

"123456789"

FIELD SELECTED

Picks `req.body.email`. Every link below runs on this value, top to bottom.

errors so far: none

← Back



Forward →

The inspector hides the `.withMessage()` calls to keep each chain short.

STEP 2 — ONE MIDDLEWARE READS EVERY RESULT

MIDDLEWARES/VALIDATE.MIDDLEWARE.JS

COPY

```
import { validationResult } from "express-validator";

export function validate(req, res, next) {
  const result = validationResult(req);

  if (result.isEmpty()) {
    return next(); // all good → go to the controller
  }

  return res.status(400).json({
    success: false,
    errors: result.array().map((err) => ({
      field: err.path,
      message: err.msg,
    })),
  });
}
```

STEP 3 – WIRE IT INTO THE ROUTE

ROUTES/AUTH.ROUTES.JS

COPY

```
import { Router } from "express";
import { registerValidator } from "../validators/auth.validator.js";
import { validate } from "../middlewares/validate.middleware.js";
import { register } from "../controllers/auth.controller.js";

const router = Router();

//                               1. run rules       2. read results  3. real work
router.post("/register", registerValidator, validate, register);

export default router;
```

Express accepts arrays of middleware, so `registerValidator` (an array of chains) can sit directly in the route.

STEP 4 – THE CONTROLLER USES MATCHEDDATA

CONTROLLERS/AUTH.CONTROLLER.JS

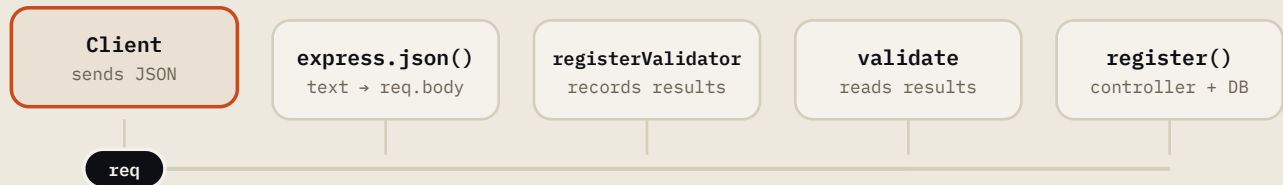
COPY

```
import bcrypt from "bcryptjs";
import { matchedData } from "express-validator";
import { User } from "../models/user.model.js";

export async function register(req, res) {
  // Only the fields we validated – already trimmed and lowercased
  const { email, phone, password } = matchedData(req);

  const hashedPassword = await bcrypt.hash(password, 10);
  await User.create({ email, phone, password: hashedPassword });

  res.status(201).json({ success: true, message: "User registered" });
}
```



raw body (just text)

```
{ "email": "123456789",  
  "phone": "abcdefghijklm",  
  "password": "  " }
```

A request arrives

At this point the body is just text sitting in the HTTP request.

← Back



Forward →

VERIFY

Our bad request now gets the same clean 400 as the hand-written version, with none of the hand-written regex:

RESPONSE

COPY

```
HTTP/1.1 400 Bad Request  
Content-Type: application/json  
  
{  
  "success": false,  
  "errors": [  
    { "field": "email", "message": "Enter a valid email address" },  
    { "field": "phone", "message": "Enter a valid 10-digit Indian mobile number" },  
    { "field": "password", "message": "Password cannot be empty or only spaces" }  
  ]  
}
```

What validationResult gives you

RAW RESULT

COPY

```
// validationResult(req).array() - exactly what express-validator 7 returns
[
  {
    "type": "field",
    "value": "123456789",
    "msg": "Enter a valid email address",
    "path": "email",
    "location": "body"
  },
  {
    "type": "field",
    "value": "abcdefghijklm",
    "msg": "Enter a valid 10-digit Indian mobile number",
    "path": "phone",
    "location": "body"
  },
  {
    "type": "field",
    "msg": "Password cannot be empty or only spaces",
    "path": "password",
    "location": "body"
  }
]
```



Don't send this raw array to the client. It contains `value` — the exact input. Without `.hide()`, a password would be echoed back into responses and logs. Map it to `{ field, message }`, the way our `validate` middleware does.

Why matchedData matters: mass assignment

Now imagine someone sends two extra fields that your form never had:

REQUEST

COPY

```
POST /api/auth/register
Content-Type: application/json

{
```

```

"email": "aman@gmail.com",
"phone": "9876543210",
"password": "Sher@2026",
"role": "admin",
"isVerified": true
}

```

Field	User.create(req.body)	User.create(matchedData(req))
<code>role: "admin"</code>	Saved, if your schema has a role field (most real apps do). The attacker is now an admin.	Dropped — it was never validated
<code>isVerified: true</code>	Saved. Verification is skipped entirely.	Dropped — isVerified stays false



Prefer to **reject** unknown fields instead of silently dropping them? Wrap your chains in `checkExact(registerValidator)`. It adds an error of type `unknown_fields` listing `role` and `isVerified`.

Beyond the body: params and query

The same idea protects route params and query strings, which never go near a Mongoose schema:

ROUTES/USER.ROUTES.JS

COPY

```

import { Router } from "express";
import { param, query } from "express-validator";
import { validate } from "../middlewares/validate.middleware.js";
import { getUser, listUsers } from "../controllers/user.controller.js";

const router = Router();

// GET /api/users/66f1c0a2b4d3e5f601234567
router.get(
  "/:id",
  param("id").isMongoId().withMessage("Invalid user id"),
  validate,
  getUser
);

```

```
// GET /api/users?page=2&limit=20
router.get(
  "/",
  query("page").optional().isInt({ min: 1 }).withMessage("page must be a positive number"),
  query("limit").optional().isInt({ min: 1, max: 100 }).withMessage("limit must be between 1 and 100"),
  validate,
  listUsers
);

export default router;
```

WHY IS MONGODB() MATTERS

COPY

```
// GET /api/users/abc - without isMongoId()
CastError: Cast to ObjectId failed for value "abc" (type string) at path "_id" for model "User"

// with isMongoId() → 400 { "errors": [{ "field": "id", "message": "Invalid user id" }]}
```



Query values are always strings — `?page=2` gives you `"2"`. After validation, convert them in the controller with `Number(req.query.page ?? 1)`.

Prefer objects? checkSchema

VALIDATORS/AUTH.SCHEMA.JS

COPY

```
import { checkSchema } from "express-validator";

export const registerSchema = checkSchema(
  {
    email: {
      trim: true,
      isEmail: { errorMessage: "Enter a valid email address" },
      toLowerCase: true,
    },
    phone: {
      trim: true,
      isMobilePhone: {

```

```

    options: ["en-IN"],
    errorMessage: "Enter a valid 10-digit Indian mobile number",
  },
},
password: {
  custom: {
    options: (value) => typeof value === "string" && value.trim().length > 0,
    errorMessage: "Password cannot be empty or only spaces",
  },
  minLength: {
    options: { min: 8 },
    errorMessage: "Password must be at least 8 characters",
  },
},
},
["body"] // only look inside req.body
);

```

Same rules, written as one object. This style feels close to Zod and Joi, which are next.



`isMobilePhone("en-IN")` also accepts `+919876543210` and `09876543210`. If you want every number stored in one format, use `.matches(/^([6-9]\d{9}$)/)` instead, or add a sanitizer that keeps the last 10 digits.

Try it yourself

Switch between the two servers and send the same data to both.

PLAYGROUND • POST /API/AUTH/REGISTER

Server without validation

Server with express-validator

email

123456789

phone

abcdefghijklm

password (shown as text so you can see the spaces)

Send request

Use the bad data

Use valid data

This demo runs in your browser and mirrors the rules from `auth.validator.js`. Try `+919876543210`, eight spaces, or an email with spaces around it.

REQUEST BODY

COPY

```
{
  "email": "123456789",
  "phone": "abcdefghijklm",
  "password": "   "
}
```

idle press "Send request"

RESPONSE

COPY

```
// response appears here
```

QUICK CHECK

`body("password").notEmpty()` — does `" "` (three spaces) pass?

A — No — `notEmpty()` rejects whitespace

B — Yes — it has length 3, so it isn't empty

QUICK CHECK

A client sends `role: "admin"` along with valid fields. What keeps it out of `User.create()`?

A — Using `matchedData(req)` instead of `req.body`

B — `validationResult(req)` automatically deletes unknown fields

06 Other options: Zod and Joi

Schema-first validation — describe the whole object once, validate it anywhere

`express-validator` attaches rules to *request fields*. Zod and Joi work differently: you describe the whole object as a **schema**, then ask “does this data match?” Neither is tied to Express — you can use them in any framework, and Zod even runs in the browser, so one schema can guard both your React form and your API.

One rule, four ways

Here's the email check from this page written in each style. Same idea, different shape:

Manual

`express-validator`

Zod

Joi

MANUAL

COPY

```
// middlewares/validateRegister.js
if (typeof email !== "string" || !EMAIL_REGEX.test(email.trim())) {
  errors.push({ field: "email", message: "Enter a valid email address" });
}
```

Zod

TERMINAL

COPY

```
npm install zod
```

VALIDATORS/AUTH.ZOD.JS

COPY

```
import * as z from "zod";

export const registerSchema = z.object({
  email: z
```

```

    .string("Email is required")
    .trim()
    .toLowerCase()
    .pipe(z.email("Enter a valid email address")),

  phone: z
    .string("Phone number is required")
    .trim()
    .regex(/^?[6-9]\d{9}$/, "Enter a valid 10-digit Indian mobile number"),

  password: z
    .string("Password is required")
    .refine((value) => value.trim().length > 0, {
      error: "Password cannot be empty or only spaces",
      abort: true, // stop here, skip the length check
    })
    .min(8, "Password must be at least 8 characters"),
});

```

MIDDLEWARES/VALIDATEBODY.JS

COPY

```

export const validateBody = (schema) => (req, res, next) => {
  const result = schema.safeParse(req.body);

  if (!result.success) {
    return res.status(400).json({
      success: false,
      errors: result.error.issues.map((issue) => ({
        field: issue.path.join("."),
        message: issue.message,
      })),
    });
  }

  req.body = result.data; // only known, cleaned fields
  next();
};

```

ROUTES/AUTH.ROUTES.JS

COPY

```
import { Router } from "express";
import { registerSchema } from "../validators/auth.zod.js";
import { validateBody } from "../middlewares/validateBody.js";
import { register } from "../controllers/auth.controller.js";

const router = Router();

router.post("/register", validateBody(registerSchema), register);

export default router;
```

`safeParse()` never throws. It returns `{ success: true, data }` or `{ success: false, error }`, and `error.issues` holds one entry per problem, each with a `path` and a `message`. Because the middleware replaces `req.body` with the parsed data, the slim controller from section 04 works unchanged.



Why `.pipe()` ? It feeds the trimmed, lowercased string into the email check, so `" Aman@Gmail.com"` passes as `"aman@gmail.com"`. And `z.object()` strips unknown keys by default, so `role: "admin"` never reaches `result.data`. Use `z.strictObject()` to reject unknown keys instead.



With TypeScript, `z.infer<typeof registerSchema>` gives you the type of the clean body for free — no separate interface to keep in sync.

Joi

TERMINAL

COPY

```
npm install joi
```

VALIDATORS/AUTH.JOI.JS

COPY

```
import Joi from "joi";

export const registerSchema = Joi.object({
  email: Joi.string().trim().lowercase().email().required().messages({
    "string.base": "Email must be text",
    "any.required": "Email is required",
    "string.empty": "Email is required",
  })
});
```

```

    "string.email": "Enter a valid email address",
  },

  phone: Joi.string().trim().pattern(/^\d{9}$/).required().messages({
    "string.base": "Phone number must be text",
    "any.required": "Phone number is required",
    "string.empty": "Phone number is required",
    "string.pattern.base": "Enter a valid 10-digit Indian mobile number",
  }),

  password: Joi.string().pattern(/S/).min(8).required().messages({
    "string.base": "Password must be text",
    "any.required": "Password is required",
    "string.empty": "Password cannot be empty or only spaces",
    "string.pattern.base": "Password cannot be empty or only spaces",
    "string.min": "Password must be at least 8 characters",
  }),
});

```

MIDDLEWARES/VALIDATEBODY.JS

COPY

```

export const validateBody = (schema) => (req, res, next) => {
  const { error, value } = schema.validate(req.body, {
    abortEarly: false, // collect every error, not just the first
    stripUnknown: true, // drop fields that aren't in the schema
  });

  if (error) {
    return res.status(400).json({
      success: false,
      errors: error.details.map((detail) => ({
        field: detail.path.join("."),
        message: detail.message,
      })),
    });
  }

  req.body = value; // only known, cleaned fields
  next();
};

```

```
// Joi's answer to the bad request (abortEarly: false)
{
  "success": false,
  "errors": [
    { "field": "email", "message": "Enter a valid email address" },
    { "field": "phone", "message": "Enter a valid 10-digit Indian mobile number" },
    { "field": "password", "message": "Password cannot be empty or only spaces" },
    { "field": "password", "message": "Password must be at least 8 characters" }
  ]
}
```

Notice that Joi reports *both* password rules, while our Zod schema stopped after the first one because of `abort: true` — and express-validator stopped because of `.bail()`. Same data, slightly different defaults. Decide which behaviour your frontend wants and configure for it.



Joi stops at the first error by default. `abortEarly: false` collects all of them, and `stripUnknown: true` removes fields that aren't in the schema.

Which one should you use?

	express-validator	Zod	Joi
Style	Chains per request field	Schema object	Schema object
Tied to Express?	Yes — built as Express middleware	No — works anywhere, including React	No — mostly used in Node.js backends
Built-in rules	All of validator.js: <code>isEmail</code> , <code>isMobilePhone("en-IN")</code> , <code>isStrongPassword</code> ...	Common formats (email, URL, UUID...); regex for the rest	A rich rule set: email, pattern, min/max, conditional rules
Cleaning data	<code>trim</code> , <code>toLowerCase</code> , <code>normalizeEmail</code> , <code>escape</code>	<code>.trim()</code> , <code>.toLowerCase()</code> , <code>.transform()</code>	Convert mode: <code>.trim()</code> , <code>.lowercase()</code>
TypeScript	Works, but no type inference	Types inferred from the schema	Typings included, no inference

	express-validator	Zod	Joi
Params and query	<code>param()</code> and <code>query()</code> built in	Pass <code>req.params</code> / <code>req.query</code> to another schema	Pass <code>req.params</code> / <code>req.query</code> to another schema
Pick it when	Plain JavaScript + Express projects	TypeScript, or one schema shared with the React form	A codebase that already uses it

KEY CONCEPT

All three solve the same problem. The **pattern** is what matters: validate at the gate, answer 400 with field-wise errors, and pass only clean, known fields to the controller. Once you understand that, switching libraries is a one-afternoon job.

QUICK CHECK

In Zod, what does `safeParse()` do when the data is invalid?

A — Throws a `ZodError` that you must catch

B — Returns `{ success: false, error }` without throwing

QUICK CHECK

Joi stops at the first error by default. Which option collects every error?

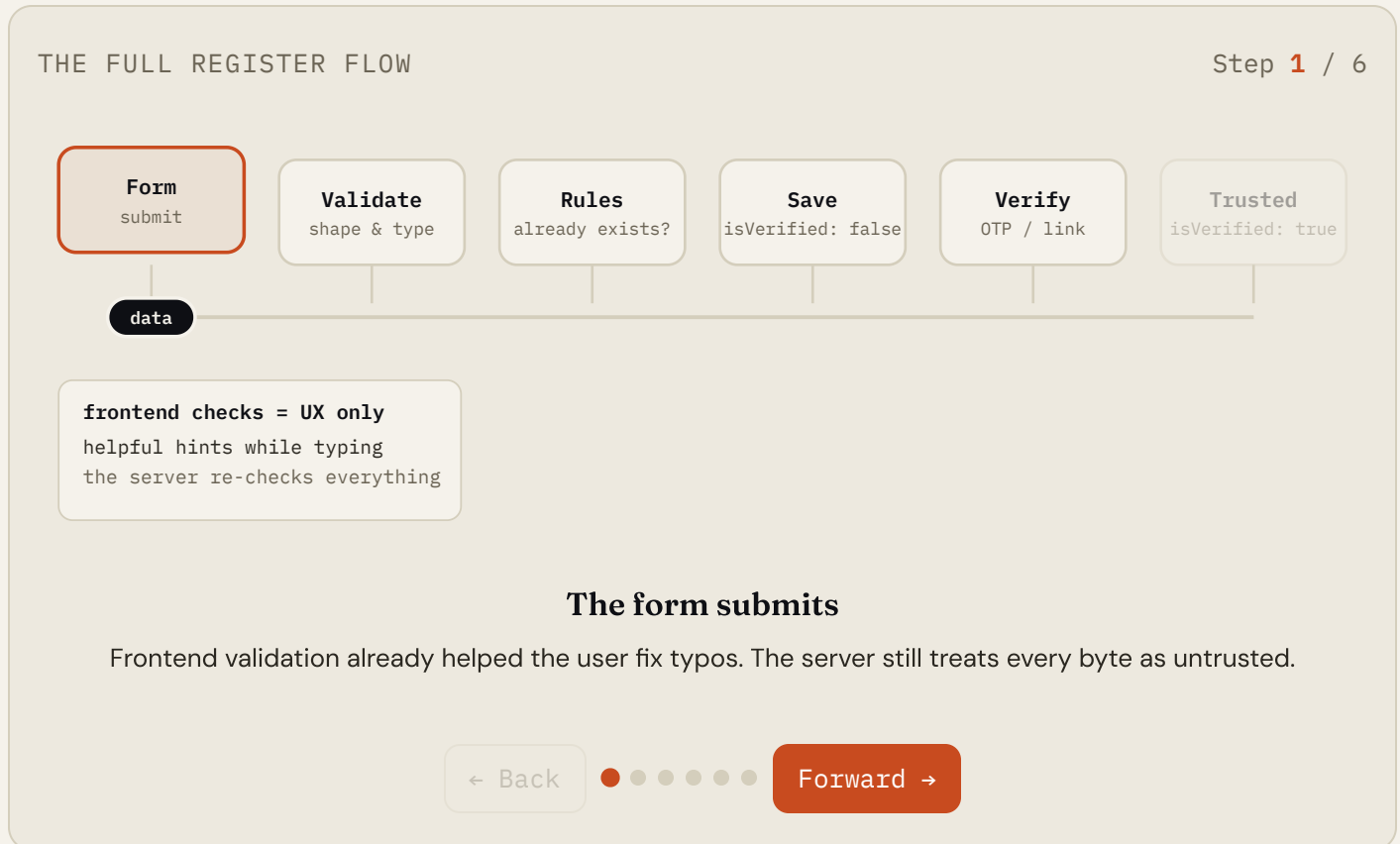
A — `abortEarly: false`

B — `allErrors: true`

07 The full picture

Where validation, business rules and verification sit in a real register flow

Let's put everything together. Validation is the first server-side step, but it isn't the only one.



CONTROLLERS/AUTH.CONTROLLER.JS — THE COMPLETE VERSION

COPY

```
import bcrypt from "bcryptjs";
import { matchedData } from "express-validator";
import { User } from "../models/user.model.js";
import { sendVerificationEmail } from "../services/email.service.js";

export async function register(req, res) {
  // 1. Validation already passed (registerValidator + validate)
  const { email, phone, password } = matchedData(req);

  // 2. Business rule: valid data can still be unacceptable
  const alreadyRegistered = await User.exists({ email });
  if (alreadyRegistered) {
    return res.status(409).json({ success: false, message: "Email already registered" });
  }
}
```

```
// 3. Save – isVerified defaults to false in the schema
const hashedPassword = await bcrypt.hash(password, 10);
const user = await User.create({ email, phone, password: hashedPassword });

// 4. Verification starts here: send a link / OTP to the email
await sendVerificationEmail(user);

res.status(201).json({
  success: true,
  message: "Account created. Check your inbox to verify your email.",
});
}
```



You can also move the “already registered” check into the chain as an async custom validator. It will then come back as a 400 through `validate` instead of a 409:

VALIDATORS/AUTH.VALIDATOR.JS – OPTIONAL

COPY

```
body("email")
  // ...the format rules from section 05...
  .bail() // never query the database for "123456789"
  .custom(async (email) => {
    const exists = await User.exists({ email });
    if (exists) throw new Error("Email already registered");
  }),
```



Keep the unique index anyway. Two sign-ups with the same email can pass `User.exists()` in the same millisecond. Only the database index stops the second one — and it fails with MongoDB error code `11000`, which you should turn into a 409:

CONTROLLERS/AUTH.CONTROLLER.JS – HANDLING THE RACE

COPY

```
const hashedPassword = await bcrypt.hash(password, 10);

try {
  await User.create({ email, phone, password: hashedPassword });
} catch (err) {
  if (err.code === 11000) {
```

```
// duplicate key: someone registered this email a moment ago
return res.status(409).json({ success: false, message: "Email already registered" })
}
throw err; // anything else is a real server error
}
```

Status codes cheat sheet

CODE	WHEN	EXAMPLE
400 Bad Request	Validation failed	123456789 is not an email
422 Unprocessable Content	Some APIs use this instead of 400 for validation errors. Pick one and stay consistent.	Same as above
409 Conflict	The input is valid but clashes with existing data	Email already registered
201 Created	Validation passed and the user was saved	"Check your inbox to verify"
403 Forbidden	Some apps use it when an unverified account tries a protected action	Unverified user tries to place an order

Common mistakes

Mistake	The fix
Validating only in React	Always validate on the server. Frontend checks are for convenience.
Forgetting validationResult(req)	Chains only record errors. Always add the validate middleware after them.
notEmpty() without trim()	" " has length 3 and passes. Trim emails and phones first; use a custom check for passwords.

Mistake	The fix
Trimming passwords	Don't. Spaces are legal password characters. Reject whitespace-only passwords instead.
<code>User.create(req.body)</code>	Use <code>matchedData(req)</code> , or the parsed value from Zod / Joi.
Sending raw error arrays	Map them to <code>{ field, message }</code> and add <code>.hide()</code> on sensitive fields.
Treating <code>unique: true</code> as validation	It's an index. Catch error code <code>11000</code> and return 409.
Sending an OTP before validating	Validate first. SMS messages cost money, and unchecked input invites spam.

THE ONE-PARAGRAPH SUMMARY

Validation checks that data looks right. **Business rules** check that valid data is acceptable.

Verification proves the data is real and belongs to the user. **Database constraints** catch whatever slips past your own code. A production API needs all four — and validation always comes first, because it's the cheapest.

QUICK CHECK

Where should `isVerified: true` come from?

A — From the request body at registration, if the frontend sends it

B — Only from your server, after the user completes the OTP or link step

QUICK CHECK

An email passes validation and the “already registered” check, but two identical sign-ups arrive in the same millisecond. What finally stops the duplicate?

A — The unique index in the database

B — The async `.custom()` check in express-validator

Sheryians Coding School · Validation Notes

Tested with Express 5.2 · express-validator 7.3 · Zod 4.6 · Joi 18 · Mongoose 9